Frontend Task F3 – Reusable DataTable Component
Team member: Vy Nguyen
AI tool: Claude (Anthropic)
Dates: Oct 22–24, 2026
Branch: f3-datatable
Repo: github.com/vnnguy30/FA26-CIS3339-Group26-Project

This is a short log of my AI conversation for F3. The prompts are in my own words, and the AI's answers are summarized. The code shown is the final version I typed into the project.

1. Choosing the task

User prompt: Any advices for me to start working on F3.

Response: The AI recommended F3 because it's one new component (DataTable.vue) used on 3 list pages. It uses props, v-for, and computed from Modules 3–4, and it's easy to demo. It also pointed out that F1 edits the same list pages, so I should coordinate with the F1 person.

2. Building DataTable.vue

User prompt: Which one should I start with building F3?.

Response: Broke F3 into steps: a basic table with columns and rows props, a "No results" row, sorting, pagination, and then using the table on the 3 pages. It explained defineProps, v-for, ref, computed, and watch as each one was added.

User prompt: (After testing Find Client) The phone and city are nested, and the rows need to open the details page.

Response: Suggested adding an optional render function to each column, plus a row-click event (defineEmits), so pages can show nested data and handle navigation.

User prompt: (Find Event) Dates show as MM/DD/YYYY.

Response: Pointed out that sorting the formatted text would put 12/01/2026 after 03/15/2027. We changed sorting to use the raw field when it exists and the displayed text otherwise.

```
const props = defineProps({
  columns: { type: Array, required: true },
  rows: { type: Array, default: () => [] }
})

const emit = defineEmits(['row-click'])
```

```
// Get the value to show: use render() if the column has one
function cellValue(row, col) {
  return col.render ? col.render(row) : row[col.field]
}

// Sorting
const sortField = ref('')
const sortDir = ref('')

function toggleSort(field) {
  if (sortField.value !== field) {
    sortField.value = field
    sortDir.value = 'asc'
  } else if (sortDir.value === 'asc') {
    sortDir.value = 'desc'
  } else {
    sortField.value = ''
    sortDir.value = ''
  }
}
```

3. Using it on the 3 list pages

User prompt: Do I need to wait for F1?

Response: No, because I work on my own branch. F1 rewrites the <script> part of each page, and I only replace the <table> in the template, so the overlap is small. It recommended telling the F1 person and keeping my change small.

I replaced each old table (about 25 lines) with the component. For example, in findclient.vue:

```
<DataTable :columns="columns" :rows="clients"
  @row-click="client => $router.push({ name: 'clientdetails', params:
{ id: client._id } })" />

columns: [
  { label: 'Name', field: 'name', render: c => c.firstName + ' ' +
c.lastName },
  { label: 'Phone number', field: 'phone', render: c =>
c.phoneNumber?.primary },
  { label: 'City', field: 'city', render: c => c.address?.city }
]
```

I did the same for findevents.vue (name, formatted date, address line 1) and findservice.vue (name, description).

4. Verification and pull request

- Tested sorting (ascending, descending, unsorted), pagination, and “No results” with 15 mock clients, events, and services shaped like the API data.
- Deleted the test page and route, restored router/index.js, and confirmed git status was clean.
- Ran npm run build, and it built with no errors.
- Checked my work against every F3 requirement in the project README.
- Opened a pull request from f3-datatable to main (no conflicts) and assigned a teammate to review it.

5. What the AI did vs. what I decided

AI helped with: explaining Vue concepts, step-by-step instructions, suggesting code for DataTable and the page changes, fixing git problems, and spotting the date-sorting issue.

I did / decided: chose F3, created and set up the team repo, typed and tested all the code, ran the build, kept the change limited to the table so it wouldn’t conflict with F1, and opened the pull request.

What I learned

How to build a reusable Vue component with props and emits, how computed and watch keep the table in sync with the data, why sorting the displayed text can be wrong (for dates), and how branches, .gitignore, and pull requests keep team work organized.